

```
type: Assignment
track: "[[General AI Fluency]]"
week: 2
phase: Foundations
status: ● Complete
tags:
  - prompt
  - upgrade
date: 14/07/2026
name: The Prompt Ladder
```

The Prompt Ladder

Task chosen

Create a prompt that helps me analyze a CTF challenge in an evidence-first way.

The goal is not to ask AI to “solve the CTF” in one shot. The goal is to make AI help me triage a challenge, reason from evidence, and decide the next useful action.

This task is personal because I work on CTF challenges in reverse engineering, pwn, forensics, and crypto. A bad prompt gives generic advice. A better prompt should help me think like a solver: inspect evidence, form a hypothesis, test it, and avoid jumping to unsupported conclusions.

Baseline

Weak baseline prompt

```
Help me solve this CTF challenge.
```

Output excerpt

```
Sure! To solve a CTF challenge, start by checking the category. If it is reverse engineering, open it in Ghidra or IDA. If it is pwn, use checksec and look for buffer overflows. If it is forensics, run strings, binwalk, and exiftool. Try searching for the flag format using grep.
```

Notes

What changed in the prompt:

Nothing yet. This is the weak baseline.

What actually improved in the output:

Almost nothing. It gave a general checklist, but not real analysis.

What still failed:

The output could apply to any CTF challenge. It did not ask for evidence, did not separate observations from guesses, and did not give a next action based on the actual artifact.

What I would try next:

Add a clearer goal so the model knows I want triage and reasoning, not a generic CTF checklist.

Version 1

Named layer added

Clearer goal

Prompt

```
Help me triage a CTF challenge and decide the next useful analysis step. Do not try to solve it immediately.
```

Output excerpt

```
To triage the challenge, first identify the category, provided files, and any visible behavior. Then determine the most likely attack surface.
```

```
Useful first steps:
```

1. List the files.
2. Identify file types.
3. Run basic metadata checks.
4. Record initial observations.
5. Decide the next tool based on those observations.

Notes

What changed in the prompt:

I changed the goal from "solve this CTF" to "triage and decide the next useful step."

What actually improved in the output:

The output stopped pretending it could solve everything immediately. It focused more on the beginning of the workflow.

What still failed:

It was still too generic. It did not match my actual CTF style or the types of challenges I work on.

What I would try next:

Add real context about my CTF workflow and challenge types.

Version 2

Named layer added

Real context

Prompt

Help me triage a CTF challenge and decide the next useful analysis step. Do not try to solve it immediately.

Context: I usually work on reverse engineering, binary exploitation, forensics, and crypto challenges. I prefer an evidence-first workflow where every conclusion is tied to something observed from the artifact, command output, decompiled code, logs, memory dump, or network behavior.

Output excerpt

Use an evidence-first triage process.

For each observation, record:

- Evidence: what you actually saw
- Meaning: what it suggests
- Confidence: low, medium, or high
- Next action: the smallest useful step to confirm or reject the hypothesis

Example:

Evidence:

checksec shows NX enabled and no stack canary.

Meaning:

A classic stack overflow may still be possible, but code execution on the stack is unlikely.

Next action:

Look for a control-flow hijack target such as ret2win, PLT/GOT reuse, or libc leak.

Notes

What changed in the prompt:

I added my real CTF context: reverse, pwn, forensics, crypto, and evidence-first reasoning.

What actually improved in the output:

The output became more specific. It started using CTF-relevant language like `checksec`, `NX`, `stack canary`, `ret2win`, `PLT/GOT`, and `libc leak`.

What still failed:

It still jumped across several possible next actions at once. I wanted one next action, not a mini-roadmap.

What I would try next:

Add a constraint that forces the model to choose only one next action.

Version 3

Named layer added

Constraints

Prompt

```
Help me triage a CTF challenge and decide the next useful analysis step. Do not try to solve it immediately.
```

```
Context: I usually work on reverse engineering, binary exploitation, forensics, and crypto challenges. I prefer an evidence-first workflow where every conclusion is tied to something observed from the artifact, command output, decompiled code, logs, memory dump, or network behavior.
```

```
Constraint: give exactly one next action at a time. Do not give a full solve plan. Wait for the result of that action before deciding the next step.
```

Output excerpt

First next action:

Run file identification on the provided artifact.

Command:

```
file ./challenge
```

Why:

Before choosing reverse engineering, pwn, forensics, or crypto tooling, we need to know what the artifact actually is.

What to send back:

Paste the full output of the file command.

Notes

What changed in the prompt:

I added a constraint: exactly one next action at a time.

What actually improved in the output:

The output became much more usable. It gave one command, explained why, and told me what result to return.

What still failed:

It became a little too rigid. If I already had basic outputs like `file`, `checksec`, or `strings`, it would still start from the beginning.

What I would try next:

Add a specified output format that lets me provide current evidence and makes the model respond based on what is already known.

Version 4

Named layer added

Specified output format

Prompt

Help me triage a CTF challenge and decide the next useful analysis step. Do not try to solve it immediately.

Context: I usually work on reverse engineering, binary exploitation, forensics, and crypto challenges. I prefer an evidence-first workflow where every conclusion is tied to something observed from the artifact, command output, decompiled code, logs, memory dump, or network behavior.

Constraint: give exactly one next action at a time. Do not give a full solve plan. Wait for the result of that action before deciding the next step.

Output format:

1. Current evidence
2. What this evidence suggests
3. What is still unknown
4. One next action
5. Why this action is the best next step
6. What result I should paste back

Output excerpt

1. Current evidence

No artifact details have been provided yet.

2. What this evidence suggests

There is not enough evidence to choose a category-specific approach.

3. What is still unknown

- File type
- Architecture
- Protections
- Input behavior
- Flag format
- Whether the challenge is local or remote

4. One next action

Run:

```
file ./challenge
```

5. Why this action is the best next step

It identifies the artifact type and prevents choosing the wrong workflow too early.

6. What result I should paste back

Paste the full output of:

```
file ./challenge
```

Notes

What changed in the prompt:

I added a required response format.

What actually improved in the output:

The answer became easier to use in a real CTF workflow. It separated evidence, interpretation, unknowns, and the next action.

What still failed:

This version also made the answer worse in one way. It became too formal for small challenges. For simple tasks, the structure may slow me down.

What I would try next:

Add quality criteria so the output stays concise and does not become a heavy report

every time.

Version 5

Named layer added

Quality criteria

Prompt

Help me triage a CTF challenge and decide the next useful analysis step. Do not try to solve it immediately.

Context: I usually work on reverse engineering, binary exploitation, forensics, and crypto challenges. I prefer an evidence-first workflow where every conclusion is tied to something observed from the artifact, command output, decompiled code, logs, memory dump, or network behavior.

Constraint: give exactly one next action at a time. Do not give a full solve plan. Wait for the result of that action before deciding the next step.

Output format:

1. Current evidence
2. What this evidence suggests
3. What is still unknown
4. One next action
5. Why this action is the best next step
6. What result I should paste back

Quality criteria:

- Be concise.
- Do not list generic tools unless they are justified by the evidence.
- Separate facts from guesses.
- State confidence when making a hypothesis.
- Prefer commands or checks that reduce uncertainty.
- If there is not enough evidence, say so directly.

Output excerpt

1. Current evidence

No challenge artifact or command output has been provided.

2. What this evidence suggests

There is not enough evidence to classify the challenge yet.

Confidence: high.

3. What is still unknown

The file type, category, architecture, protections, and expected input are unknown.

4. One next action

Run:

```
file ./challenge
```

5. Why this action is the best next step

It gives the first hard fact about the artifact and avoids guessing the workflow.

6. What result I should paste back

Paste the full output of the command.

Notes

What changed in the prompt:

I added quality criteria: concise, evidence-based, no generic tool lists, separate facts from guesses, state confidence, and reduce uncertainty.

What actually improved in the output:

The output became tighter. It no longer dumped a long list of tools. It admitted that there was not enough evidence and gave one justified first step.

What still failed:

The prompt is still slightly biased toward file-based challenges. Some CTF tasks start with a remote service, website, packet capture, or source code snippet.

What I would try next:

Turn the final prompt into a reusable template where the user can specify the category, artifact, current evidence, and constraints.

Final reusable prompt

You are helping me analyze a CTF challenge using an evidence-first workflow.

Goal:

Help me decide the next useful analysis step. Do not try to solve the whole challenge immediately.

Challenge context:

- Category: [reverse / pwn / forensics / crypto / web / misc / unknown]
- Artifact or target: [file name, remote service, source code, PCAP, memory dump, website, logs, etc.]
- Flag format: [FLAG FORMAT IF KNOWN]
- Current evidence:
[PASTE COMMAND OUTPUTS, OBSERVATIONS, DECOMPILED CODE, LOGS, ERROR MESSAGES, OR SCREENSHOTS HERE]

Workflow rules:

- Use only the evidence I provided.
- Separate facts from guesses.
- Tie every conclusion to a specific observation.
- Give exactly one next action.
- Do not give a full solve plan.
- Do not list generic tools unless the evidence justifies them.
- If there is not enough evidence, say so clearly.
- Prefer the next action that reduces uncertainty the most.

Output format:

1. Current evidence
2. What this evidence suggests
3. Confidence level
4. What is still unknown
5. One next action
6. Why this action is the best next step
7. What result I should paste back

Quality criteria:

- Be concise.
- Be technical.
- Avoid filler.
- Do not hallucinate challenge details.
- Do not assume the vulnerability or solution too early.
- Work incrementally from evidence.

Final reflection

The biggest improvement came from adding the evidence-first context, the one-action constraint, and the quality criteria.

The baseline gave generic CTF advice. The clearer goal made the model focus on triage. Real context made it sound like a real reverse/pwn/forensics workflow. The one-action constraint was the most useful change because it stopped the model from dumping a full solve plan.

The specified output format helped organize the reasoning, but it also made the answer too formal for simple challenges. The quality criteria fixed part of that by forcing the answer to stay concise.

The final prompt works for a stranger because they can paste their own category, artifact, evidence, and flag format. It does not depend on my private context.